



**Department of Electrical,  
Computer, & Biomedical Engineering**  
Faculty of Engineering & Architectural Science

## COE/ELE 70AB Milestones Compliance Report (MCR)

|                                           |                                                                                                                                                                                                                        |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Title</b>                      | Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming<br><br>Optimizing Collision Detection with the GPU for Lagrangian (Particle Based) Fluid Simulations for Real-Time Gaming Applications in Unity |
| <b>MCR Number</b>                         | 2                                                                                                                                                                                                                      |
| <b>Project Manager for the MCR period</b> | Student D: Davis Cheung                                                                                                                                                                                                |
| <b>Team Players for the MCR period</b>    | Names of the rest of the team other than the project manager:<br>Student A: Victor Do<br>Student B: Jj Marr<br>Student C: Cameron Tuffner-Lyons                                                                        |
| <b>Faculty Supervisor</b>                 | Your FLC Name:<br>Gul Khan                                                                                                                                                                                             |

1. **Tasks Outlined for the Reporting Period (e.g. MCR I – Weeks 4 & 5)** (Provide detailed information on the tasks to be completed for the reporting period as per the milestone submitted to your FLC in Week3)

**Group: GK02**

**Student A:**

1. Decompose Research Papers into Workable Tasks
2. Break down complex concepts into manageable components.
3. Identify the core ideas or algorithms within each research paper and break them into smaller, executable tasks (e.g., implementing specific optimization methods).
4. Smoothed Particles
  - a. Task: Refine the smoothing techniques used for particles in simulation.
  - b. Test different smoothing kernels to reduce artifacts in particle interactions.
5. Optimize kernel functions for GPU processing to enhance real-time performance.

**Student B:**

1. Assign task types.

2. Separate tasks into “Research,” “Prototype Development,” and “Optimization” categories.
3. Research Task: Study spatial subdivision algorithms and their impact on reducing unnecessary collision calculations.
4. Development Task: Implement basic spatial subdivision in Unity.
5. Optimization Task: Profile and test the implementation on both CPU and GPU.

**Student C:**

1. Timeline and priority.
  - a. Set priority levels for each task and assign deadlines to make progress systematic.
2. Link tasks to the overall project.
  - a. Ensure each task directly contributes to the end goal of optimizing collision detection for fluid simulations.
3. The Interpolation Equation
  - a. Fine-tune interpolation methods to improve accuracy.
  - b. Explore different interpolation techniques for particle properties like velocity and pressure.
  - c. Implement parallelized interpolation on the GPU.

**Student D:**

1. Reevaluation and Optimization of Key Fluid Simulation Components
  - a. Reevaluate the gravity force implementation, ensuring realistic interaction with particle-based collisions.
  - b. Investigate gravitational force formulas used in existing simulations.
  - c. Test GPU acceleration for gravity-based interactions between particles.
2. Gradient Calculations
  - a. Identify potential optimizations for gradient-based interactions in fluid particles.
  - b. Profile GPU-based gradient calculation for improved performance.

2. **Progress Made in Reporting Period (e.g. MCR I – Weeks 4 & 5)** (Provide detailed information on the progress that you (as a group and individual) made during the reporting period. You can include figures, datasheets, flowcharts etc. and additional information as requested by your FLC. You should use your progress to justify compliance to the tasks outlined for the reporting period as per the milestones submitted to your FLC in Week3)

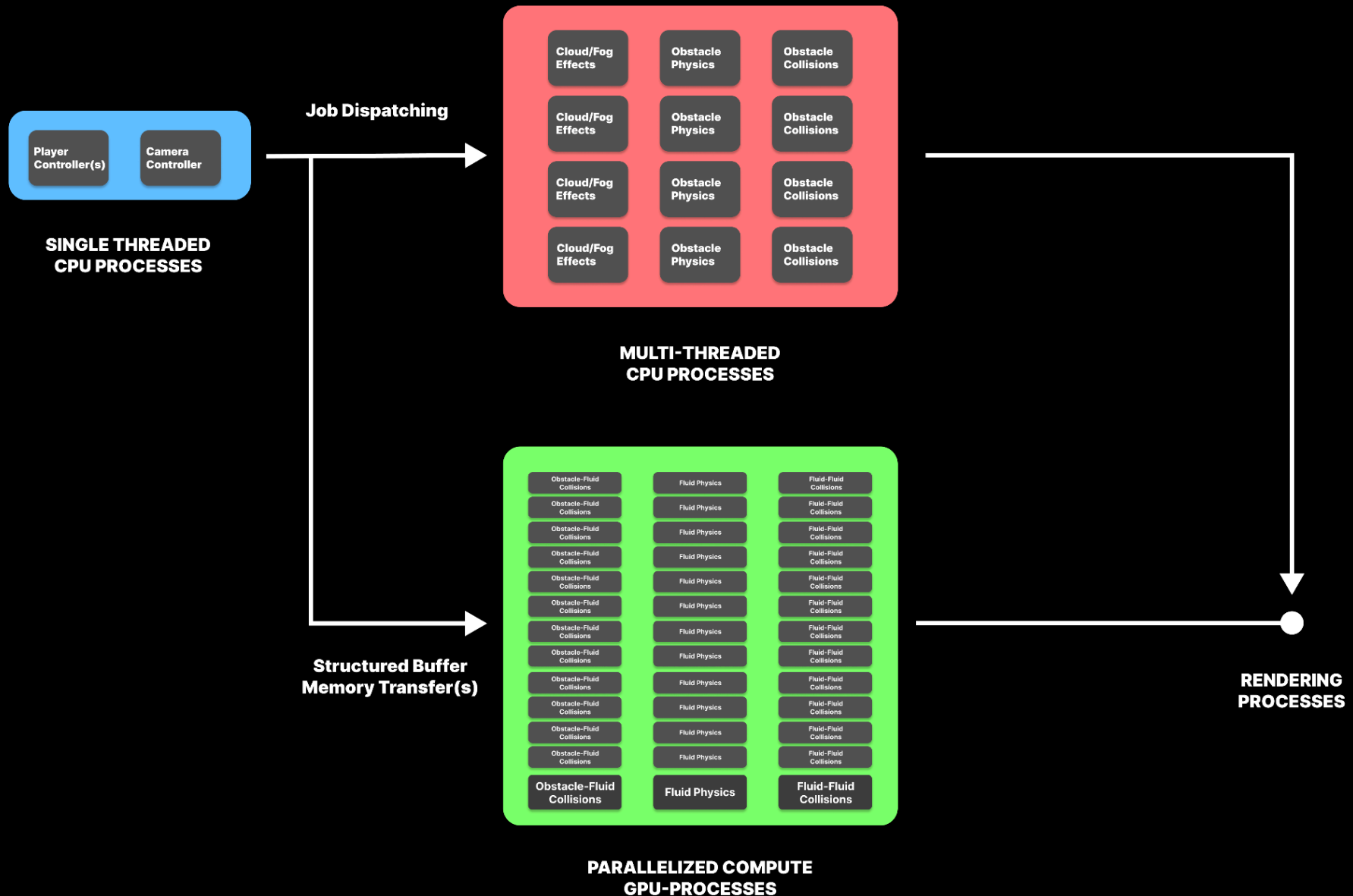
**Group:**

**Student A:**

- Worked on github page
- Clarified tasks to do in internal documents
- Worked on Prototypes:
  - CPU multithreading for physics
  - GPU fluid simulation
  - Combined CPU-GPU implementations into demo scene
  - Added basic camera controller
  - Added basic UI elements
- Researched optimizations for collision detection
- Researched implementations of fluid simulators
- Worked on CPU-GPU workload distribution and profiling analysis

On the next page is a simplified diagram that shows how the CPU-GPU workload can be split between CPU threads and GPU workgroups.

## OPTIMAL CPU-GPU WORK DISTRIBUTION for Fluid Simulation in Games



**Student B:**

I set up GitHub issues to remind people to write the milestone compliance report. I also created a GitHub issue reflecting the current status of the CPU-GPU implementation.

**Student C:**

- Created a timeline for the project indicating which tasks should be completed.
  - also decided on the priority of each task to ensure the more critical tasks are completed before others.
  - Each task revolves around certain elements of the project such as
    - The collision detection algorithms
    - The physics algorithms for simulating the fluid
    - UI elements for interacting with the game/simulation
    - Scene creation for demonstrating the capabilities and performance of the CPU + GPU implementation vs only GPU.

| Task                                                                                                                                                                                                                    | Priority | Timeline for ICompletion | Task Group                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------------------------|--------------------------------------------|
| Develop a sphere in Unity that can interact with physical boundaries such as a square.                                                                                                                                  | High     | Week 7                   | Basic                                      |
| Develop a method for generating particles (spheres) and determine how to have them interact such that they can approximate a fluid i.e using SPH fluid physics                                                          | High     | Week 7-8                 | collision detection and physics algorithms |
| Implement the fluid physics algorithm (smoothed-particle hydrodynamics) inside a Unity scene using multithreaded CPU to evaluate how the simulation performs both for fluid simulation and processing time/utilization. | High     | Week 7-8                 | collision detection and physics algorithms |
| If the fluid simulation is evaluation does not look good or run well then optimize the physics algorithms to improve the simulation                                                                                     | High     | Week 7-9                 | collision detection and physics algorithms |
| Implement Smoothed-Particle                                                                                                                                                                                             | High     | Week 7-9                 | collision detection                        |

|                                                                                                                                                                                                                                 |        |            |                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|------------|--------------------------------------------------------------------------|
| Hydrodynamics in a Unity scene using the GPU. Evaluate the performance on GPU for both fluid simulation and processing time/utilization.                                                                                        |        |            | and physics algorithms                                                   |
| Compare the optimized CPU and optimized GPU implementations and compare performance to determine a baseline.                                                                                                                    | High   | Week 9     | collision detection and physics algorithms                               |
| Determine the best method for combining CPU and GPU processing for collision detection and fluid simulation. Focusing on the minimization of data transfer and proper load balancing/division of labor to maximize utilization. | High   | Week 10-12 | collision detection and physics algorithms                               |
| Implement the collision detection and fluid simulation for CPU-GPU heterogeneous computation.                                                                                                                                   | High   | Week 12-13 | collision detection and physics algorithms                               |
| Determine which UI elements are to be implemented to provide interaction and certain abilities to modify the simulation                                                                                                         | Normal | Week 8-9   | UI elements for interacting with the simulation.                         |
| Implement the UI features in a Unity Scene for functionality testing                                                                                                                                                            | Normal | Week 9-10  | UI elements for interacting with the simulation.                         |
| Test UI Functionality with CPU or GPU implementation and debug as needed. Update UI if certain desired features are missing.                                                                                                    | Normal | Week 10-11 | UI elements for interacting with the simulation.                         |
| Determine scenarios that would best demonstrate the fluid simulation. Focusing on scenarios that are very computationally heavy.                                                                                                | Low    | Week 9-12  | Scene creation for demonstrating capabilities of CPU-GPU implementation. |
| Create the Unity scenes that will best demonstrate the fluid-simulation.                                                                                                                                                        | Low    | Week 12-13 | Scene creation for demonstrating capabilities of CPU-GPU                 |

|  |  |  |                 |
|--|--|--|-----------------|
|  |  |  | implementation. |
|--|--|--|-----------------|

- Investigated particle interpolation and the feasibility for implementing in our project.
  - Consistent Shepard Interpolation
    - The Shepard interpolation is used to reduce errors in the SPH fluid calculations that can appear due to irregularly distributed particles.
    - The Shepard correction does this by computing a correction factor to apply to the SPH regions where these irregularities occur. This should reduce the number of physics artifacts.
    - The paper linked below covers the methods for implementing Shepard correction as well as certain corrections to account for rigid boundaries.
    - [https://www.hdm-stuttgart.de/hochschule/forschung/forschungsschwerpunkte/digital\\_media/publikationen/Consist\\_Sh\\_Mat/Consistent\\_Shepard.pdf](https://www.hdm-stuttgart.de/hochschule/forschung/forschungsschwerpunkte/digital_media/publikationen/Consist_Sh_Mat/Consistent_Shepard.pdf)

#### **Student D:**

- Collision detection & general GPU algorithm research
  - Read and took notes on “Chapter 32: Broad Phase Collision Detection” of *GPU Gems 3*  
(<https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-32-broad-phase-collision-detection-cuda>)
    - Wide variety of parallel algorithms ; of interest: Parallel radix sort, parallel spatial sub-division
    - (primarily targeting GPU-only platforms and CUDA; generally applicable)
  - Read “Parallel Scan for Stream Architectures”, a paper on parallel algorithms (specifically scan, largely GPU-focused;  
<https://libraopen.lib.virginia.edu/downloads/6t053g00z>)
    - Parallel scan algorithms in general (parallelizing the summation of series of values)
  - Read “Single-pass Parallel Prefix Scan with Decoupled Look-back”, a specific parallel implementation of prefix scan  
([https://research.nvidia.com/sites/default/files/pubs/2016-03\\_Single-pass-Parallel-Prefix/nvr-2016-002.pdf](https://research.nvidia.com/sites/default/files/pubs/2016-03_Single-pass-Parallel-Prefix/nvr-2016-002.pdf))
    - A specific parallel scan algorithm - “prefix scan with decoupled look-back”
  - Algorithms to consider implementing for broad-phase detection:
    - Radix sort (Easy solution; avg  $\ln n$  perf)
    - Counting sort (w/ parallel prefix scan; allegedly better performance than radix sort w/ approx same avg of  $\ln n$ )
- Alternative CPU-GPU collision detection approaches

- Density-based dispatching
  - Assumption: Hash-based broad-phase collision detection (spatial sub-division)
  - Concept: dispatch narrow-phase collision detection for hashed areas w/ low fluid density (few collisions) to CPU, and high-density hashed areas to GPU
  - Use a heuristically-obtained constant value to determine density (# particles in a grid) threshold

3. **Difficulties Encountered in Reporting Period** (Provide detailed information on the difficulties and issues that you encountered during the reporting period and how you plan to address this in the following periods)

**Group:**

**Student A:**

It was midterm season, so much time was spent studying to achieve a good grade.

**Student B:**

I had too many lab reports due and several midterms. This made it difficult to perform work on the Capstone project.

**Student C**

Please read *Difficulties Encountered in Report Period: Student A; Student B*.

**Student D:**

Please read *Difficulties Encountered in Report Period: Student A; Student B*.

4. **Tasks to Be Completed in the Next Reporting Period** (Outline the tasks to be completed in the next reporting period. Please note this should match with your milestones submitted to your FLC in Week3, however in consultation with (and approval of) your FLC, you can modify this to accommodate incomplete tasks from previous period. Here you should also identify the Project Manager for the next period)

**Group:**

**Student A:**

**Spatial Grid Code**

Task: Refactor and improve spatial grid-based particle partitioning.

Test different spatial partitioning techniques (e.g., uniform grid vs. octree).

Compare performance of GPU-based spatial grids.

**Optimizing Particle Lookups**

Task: Optimize the lookup process for particle interactions.

Research spatial partitioning and hashing techniques.

Implement particle lookup optimizations in Unity using GPU acceleration.

**Student B:**

Because I was unable to effectively do anything during this reporting period,

1. Assign task types.
2. Separate tasks into “Research,” “Prototype Development,” and “Optimization” categories.
3. Research Task: Study spatial subdivision algorithms and their impact on reducing unnecessary collision calculations.
4. Development Task: Implement basic spatial subdivision in Unity.
5. Optimization Task: Profile and test the implementation on both CPU and GPU.

**Student C:**

**Mouse Force**

**Task:** Refine user interaction forces (e.g., mouse-based fluid manipulation).

Profile the impact of mouse force on particle interactions and optimize for real-time feedback.

**Artificial Viscosity**

**Task:** Reevaluate artificial viscosity implementation.

Test various viscosity models to minimize artifacts in fluid simulations.

Profile GPU-accelerated viscosity calculations.

**Student D:**

- **The Pressure Force**
  - Task: Revisit pressure force calculations.
  - Test different models for simulating pressure forces between fluid particles.
  - Experiment with optimizations for high particle counts in real-time scenarios.
- **Pressure Problems**
  - Task: Investigate issues related to pressure force inaccuracies.
  - Debug and optimize the pressure force algorithms to improve simulation realism.
- **Bugs**
  - Task: Perform a thorough review of known bugs and unresolved issues.
  - Set up a systematic approach to fixing bugs related to particle interactions, GPU integration, and parallelism.